# Ders 26 Çalışma Özeti

## Ders 26 Çalışma Özeti

### Genel Konular

- Karşılaştırma yapmadan sıralama: sayarak sıralama (counting sort).
- Karşılaştırmalı sayma (comparison counting sort).
- Dağılım sayımı (distribution counting sort).
- Basamak sıralama (radix sort): LSD (least significant digit) ve MSD (most significant digit).
- Sıralama algoritmalarını karşılaştırma ölçütleri: performans, ek bellek (in-place / out-of-place), kararlılık (stable).
- Algoritma tasarım yaklaşımlarına göre sıralama yöntemleri ve karşılaştırma tablosu.
- Dönemin son dersi: proje, mazeret sınavı ve final duyuruları.

### Hocanın Özellikle Vurguladığı Kısımlar

- Counting sort hiçbir anahtar karşılaştırması (`if`) yapmadan, yalnız sayarak sıralar. Karşılaştırmanın pahalı olduğu bir mimaride işe yarar.
- Counting sort doğrusal zamanda çalışır ama bedeli ek alandır: sayaç dizisi ve girdi kadar bir çıktı dizisi gerekir. Dizi yerinde sıralanamaz.
- Counting sort, farklı değer sayısı az, tekrar sayısı çok olan verilerde idealdir. Örnek: ilkokul öğrencilerini yaşa göre sıralamak (5-11 arası değerler, milyonlarca tekrar).
- Radix sort her basamakta counting sort yapar. Counting sort kararlı olduğu için önceki basamaklardaki sıra korunur.
- LSD, sabit uzunluklu anahtarlar için biçilmiş kaftandır: IBAN, T.C. kimlik numarası, GUID, veritabanı indeksleri.
- MSD değişken uzunluklu girdilerde (sözlük gibi) avantajlıdır; karakterlerin çoğuna hiç bakmadan sıralayabilir. Parçalar çok küçükse yavaşlar; bu durumda küçük parçalar quicksort veya insertion sort ile sıralanır.
- MSD'de her parça bağımsız olduğu için paralel çalıştırılabilir.
- Sıralama algoritması seçimi bir ödünleşimdir (trade-off): karmaşıklık, ek alan ve kararlılık birlikte düşünülür.

### Kısa Tekrar Notları

- Counting sort: $O(n + k)$ zaman, $O(n + k)$ ek alan. $k$: değer aralığının boyu.
- Comparison counting: her eleman kendinden sonrakilerle karşılaştırılır. $n(n-1)/2$ karşılaştırma, $O(n^2)$.
- Distribution counting: değerler $[l, u]$ aralığındaysa $D[A[i] - l]$ sayılır, birikimli toplam alınır, sondan başa yerleştirilir.
- LSD radix sort: en sağ basamaktan sola doğru her basamakta kararlı counting sort. $W$ basamak için $O(W(n + R))$.
- MSD radix sort: en soldaki basamaktan başlar, her harf grubunu özyinelemeli olarak sıralar. Uzunlukların eşit olması gerekmez.
- Kararlı (stable): eşit anahtarlı elemanların girdideki sırası çıktıda korunur.
- Yerinde (in-place): yalnız sabit (veya küçük) ek alan kullanılır.

## Detaylı Açıklamalar

### Counting Sort

```
COUNTING-SORT(A, B, k)
  C[0..k] = 0                        // O(k)
  her j = 0..n-1 için C[A[j]]++      // O(n): sayma
  her i = 1..k için C[i] += C[i-1]   // O(k): birikimli toplam
  her j = n-1..0 için                // O(n): sondan başa yerleştirme
    C[A[j]]--
    B[C[A[j]]] = A[j]
```

- Örnek dizi: `3 5 2 2 3 1 4`.
- Sayma sonrası: 1 → 1 tane, 2 → 2, 3 → 2, 4 → 1, 5 → 1.
- Birikimli toplam: 1, 3, 5, 6, 7. Bu değerler her değerin sıralı dizide nerede biteceğini gösterir.
- Sondan başa yerleştirince: `1 2 2 3 3 4 5`.
- Sondan başa gitmek eşit elemanların sırasını korur; algoritmayı kararlı yapan budur.
- Toplam: $O(2n + 2k) = O(n + k)$.
- Bir elemandan küçük 10 eleman varsa, o eleman sıralı dizide 11. sırada (indis 10) durur.

### Comparison Counting Sort

```
COMPARISON-COUNTING-SORT(A)
  Count[0..n-1] = 0
  her i = 0..n-2 için
    her j = i+1..n-1 için
      eğer A[i] < A[j] ise Count[j]++
      değilse Count[i]++
  her i = 0..n-1 için S[Count[i]] = A[i]
```

- Örnek: `62 31 84 96 19 47`.

| Adım | Count dizisi |
|---|---|
| Başlangıç | 0 0 0 0 0 0 |
| $i = 0$ (62) | 3 0 1 1 0 0 |
| $i = 1$ (31) | 3 1 2 2 0 1 |
| $i = 2$ (84) | 3 1 4 3 0 1 |
| $i = 3$ (96) | 3 1 4 5 0 1 |
| $i = 4$ (19) | 3 1 4 5 0 2 |

- Count değerleri her elemanın çıktıdaki indisidir: 62 → 3, 31 → 1, 84 → 4, 96 → 5, 19 → 0, 47 → 2.
- Sonuç: `19 31 47 62 84 96`.

### Distribution Counting Sort

- Değerler $l \leq A[i] \leq u$ aralığındaki tam sayılardır.
- Frekanslar $D$ dizisinde tutulur, birikimli toplama çevrilir. İstatistikte birikimli frekansa dağılım denir.
- En küçük değer $l$'ye eşit elemanlar $0$ ile $F[0] - 1$ arasına, $l + 1$'e eşit olanlar $F[0]$ ile $F[0] + F[1] - 1$ arasına yerleşir.

```
DISTRIBUTION-COUNTING-SORT(A, l, u)
  D[0..u-l] = 0
  her i = 0..n-1 için D[A[i] - l]++
  her j = 1..u-l için D[j] += D[j-1]
  her i = n-1..0 için
    j = A[i] - l
    S[D[j] - 1] = A[i]
    D[j]--
```

- Örnek: `13 11 12 13 12 12`, aralık 11-13.
- Frekans: 11 → 1, 12 → 3, 13 → 2. Birikimli: `1 4 6`.

| İşlenen $A[i]$ | $D$ (önce) | Yazılan indis | $D$ (sonra) |
|---|---|---|---|
| $A[5] = 12$ | 1 4 6 | 3 | 1 3 6 |
| $A[4] = 12$ | 1 3 6 | 2 | 1 2 6 |
| $A[3] = 13$ | 1 2 6 | 5 | 1 2 5 |
| $A[2] = 12$ | 1 2 5 | 1 | 1 1 5 |
| $A[1] = 11$ | 1 1 5 | 0 | 0 1 5 |
| $A[0] = 13$ | 0 1 5 | 4 | 0 1 4 |

- Sonuç: `11 12 12 12 13 13`.
- Bu, dersin başındaki counting sort ile aynı algoritmadır; yalnız $l$ kadar kaydırılmış indis kullanır.

**LSD Radix Sort**

- Tarihçe: Herman Hollerith'in delikli kart sıralama makinesi. Kartlar kolon kolon sıralanır. Bu makine ABD nüfus sayımının sonuçlarını yıllar yerine aylar içinde çıkardı ve IBM'in temelini attı.
- Girdi: $n$ tane, her biri $W$ karakterli dizgi; alfabe $R$ sembollü.
- Algoritma: $d = W - 1$'den $0$'a kadar, $d$. karaktere göre kararlı counting sort yap.
- Örnek 1: `CAT HIM HEM BET`

| Adım | Sıra |
|---|---|
| Başlangıç | CAT, HIM, HEM, BET |
| 3. harfe göre | HIM, HEM, CAT, BET |
| 2. harfe göre | CAT, HEM, BET, HIM |
| 1. harfe göre | BET, CAT, HEM, HIM |

- Ders sonrası not: Bu kelimelerle 2. harf adımının kararlı sonucu CAT, HEM, BET, HIM olur. Hoca bu adımda ikinci harfleri "I, A, A, A" diye okudu; slayttaki kelimeler büyük olasılıkla farklıydı (örneğin HAM, BAT).
- Örnek 2 (sayılarla): `318 457 417 218`

| Adım | Sıra |
|---|---|
| Birler basamağı | 457, 417, 318, 218 |
| Onlar basamağı | 417, 318, 218, 457 |
| Yüzler basamağı | 218, 318, 417, 457 |

- Oklar birbirini kesmez: eşit karakterli elemanlar girdideki sıralarını korur.
- 85 milyon T.C. kimlik numarası (11 basamak): karşılaştırmalı sıralama yaklaşık $n \log_2 n \approx 2.2$ milyar adım ister. LSD yaklaşık $11 \times 85$ milyon $\approx 935$ milyon adımda bitirir.

**MSD Radix Sort**

- En soldaki karakterden başlar. Her karaktere göre counting sort yapar, girdiyi alfabe kadar parçaya böler.
- Her parçayı bir sonraki karaktere göre özyinelemeli olarak sıralar.
- Quicksort'a benzer: quicksort diziyi 3 parçaya (küçükler, pivot, büyükler) böler; MSD alfabe kadar parçaya böler.
- Kelime bittiğinde, kelimenin sonu (null karakter) her harften küçük kabul edilir: `all`, `allocate`'den önce gelir.
- Algoritmadaki iyileştirme: parça boyutu alfabe boyutundan küçükse MSD yerine doğrudan quicksort (veya insertion sort) kullan.

- Örnek (3 harfli 12 kelime): ilk harfe göre A 0'dan, B 2'den, C 5'ten, D 6'dan, E 8'den, F 9'dan başlar. Sonra her grup kendi içinde sıralanır. Tek elemanlı gruplara hiç girilmez.
- Sıralı sonuç: `ace add bad bed bee cab dab dad ebb fad fed fee`.
- Bu örnekte 24 karakterin yaklaşık 19'una bakıldı. Değişken uzunluklu sözlük örneğinde karakterlerin yalnız yaklaşık %30'una bakmak yetti.
- Kitap örneği (she sells seashells…): sonuç `are by sea seashells seashells sells sells she she shells shore surely the the`.
  - `seashells` ile `seashells` gibi aynı kelimelerde son karaktere kadar her karaktere bakmak gerekir.
- Kötü yanları:
  - Çok sayıda küçük parça oluşursa sürekli özyinelemeli çağrı ve sayaç sıfırlama nedeniyle yavaşlar.
  - Ek alan: çıktı dizisi ve her özyineleme düzeyi için sayaç dizisi ($n+DR$, $D$: özyineleme derinliği).

**Sıralama Algoritmalarını Karşılaştırma Ölçütleri**

- **Performans:** çalışma süresi karmaşıklığı. Karşılaştırma ve yer değiştirme sayısı ya da dizi erişim sayısı ile ölçülür.
- **Ek bellek:**
  - Yerinde (in-place): rafta kitapları yalnız iki elinizle sıralamak gibi. Sabit ek değişken veya küçük bir çağrı yığını.
  - Yerinde olmayan (out-of-place): kitapları başka bir rafa dizerek sıralamak. Counting sort ve merge sort örnek.
- **Kararlılık:** Eşit anahtarlı elemanların sırası korunuyor mu? Örnek: 5 yaşındaki iki öğrenciden önce sıraya giren, sıralamadan sonra da önde kalmalı.

**Tasarım Yaklaşımlarına Göre Sıralama**

- Kaba kuvvet (brute force): selection sort, bubble sort.
- Azalt ve yönet (decrease and conquer): insertion sort.
- Dönüştür ve yönet (transform and conquer): heap sort. Girdi heap'e dönüştürülür.
- Böl ve yönet (divide and conquer):
  - Merge sort: diziyi ortadan ikiye böl, tek elemanlı parçalara in, sıralı parçaları birleştir. $O(n \log n)$, $n$ boyutunda ek dizi gerekir.
  - Quicksort: pivot seç, küçükleri ve büyükleri ayır, her parçayı özyinelemeli sırala. Ortalama $O(n \log n)$, en kötü $O(n^2)$, $c \log n$ ek alan.
- Quicksort, 20. yüzyılın en iyi 10 algoritmasından biri seçilmiştir.

**Karşılaştırma Tablosu**

| Algoritma | Kararlı | Yerinde | Zaman (yaklaşık) | Ek alan |
|---|---|---|---|---|
| Selection sort | Hayır | Evet | $n^2/2$ | 1 |
| Insertion sort | Evet | Evet | $n^2/2$ (en kötü) | 1 |
| Shell sort | Hayır | Evet | yaklaşık $n^{3/2}$ | 1 |
| Merge sort | Evet | Hayır | $n \lg n$ | $n$ |
| Quicksort | Hayır | Evet | $1.39\, n \lg n$ | $c \lg n$ |
| Heap sort | Hayır | Evet | $2n \lg n$ | 1 |
| LSD radix sort | Evet | Hayır | $2nW$ | $n + R$ |
| MSD radix sort | Evet | Hayır | $2nW$ (en kötü) | $n + DR$ |

- $W$: anahtar uzunluğu, $R$: alfabe boyutu.
- Alfabe çok büyükse radix sort avantajını kaybeder. Küçük alfabe ve çok tekrar varsa büyük kazanç sağlar.

**Ders sonrası notlar**

- **Bubble sort kararlıdır:** Hoca bubble sort'un sırayı korumadığını söyledi. Yalnız $A[j] > A[j+1]$ olduğunda yer değiştiren standart bubble sort kararlıdır. Kararsız olanlar selection sort, shell sort, quicksort ve heap sort'tur.
- **Quicksort açıklaması:** Hoca "merge sort gözetir, quicksort gözetmez" dedikten sonra "konuma bağlı olarak yaptığı için gözetiyor" diye ekledi. Doğrusu: merge sort konuma göre böldüğü için kararlıdır; quicksort değere (pivota) göre bölüp uzak elemanları yer değiştirdiği için kararlı değildir.
- **LSD hesabı:** Hoca 11 basamak × 85 milyon için "850 milyon adım" dedi. Doğrusu $11 \times 85$ milyon $= 935$ milyon.
- **LSD karmaşıklığı:** Hoca "basamak sayısı artı sigma" dedi. Doğrusu her basamak için bir counting sort yapıldığı için $O(W(n+R))$'dir.
- **Counting sort'taki $k$:** Hoca $k$'yı "farklı değer sayısı" diye tanımladı. Sayaç dizisinin boyu, değer aralığının boyudur ($u - l + 1$). Örneğin değerler 1 ve 1000000 ise yalnız 2 farklı değer olsa da 1000000 boyutlu sayaç gerekir.
- **"Döngü koşulu da bir karşılaştırma değil mi?" sorusu:** Döngü sayacının kontrolü vardır. Counting sort'u karşılaştırmasız yapan, elemanların anahtarlarının birbiriyle karşılaştırılmamasıdır. Bu yüzden karşılaştırmalı sıralamanın $\Omega(n \log n)$ alt sınırı ona uygulanmaz.
- **Kitap adı:** Altyazıda "Sevgi Ebit'in kitabı" diye geçen kaynak, büyük olasılıkla Sedgewick ve Wayne'in *Algorithms* kitabıdır.

## Duyurular

- Bir öğrenci için mazeret sınavı ertesi gün saat 16:00'da Online Yıldız üzerinden yapılacak.
- Proje teslimi 30 Mayıs akşamı (19:00 civarı).
- Final sınavı pazartesi, online, vizeye benzer biçimde.
- Ödev ve proje notları finale kadar OBS'ye girilecek.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.